Recurente de complexitate

-> folosite pentru calculul algoritmilor recursivi

```
int f ( int n)
{
        if (n == 1 || n == 0)
                return 1;
        ...
         return 2*f(n/2) + 3*f(n/4);
        // return f(n-1) + f(n-2);
}
```

O(Phi^n) -> phi = (sqrt(5)+1)/2 ~= 1.61 = O(2^n)

T(n) = 2 T(n/2) + 3T(n/4) + O(1) -> exemplu de recruenta

```
MergeSort(int v[], int i, int j)
{
        if (i == j)
                return;
        MergeSort(v,i,(i+j)/2);
        MergeSort(v,(i+j)/2+1, j);
        Merge(v,i,j); // Theta(n)
}
```

T(n) = 2 * T(n/2) + Theta(n)
pot scrie si
T(n) = 2 * T(n/2) + c* n
pentru simplitate, vom lucra cu
T(n) = 2 * T(n/2) + n

Complexitate MergeSort -> Theta(n log n)

1. Metoda iterativa

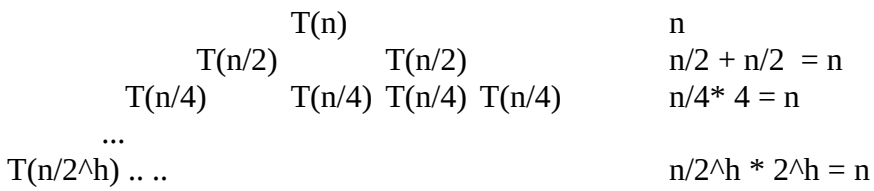
T(n) = 2 * T(n/2) + n
T(n/2) = 2* T(n/4) + n/2 | * 2
T(n/4) = 2* T(n/8) + n/4 | * 2^2
...
T(n/2^k) = 2* T(n/2^(k+1)) + n/2^k | * 2^k

n/2^(k+1) = 1
=> 2^(k+1) = n =>
k+1 = log n

T(n) = ~~2 * T(n/2)~~ + n
~~2* T(n/2)~~ = ~~2^2T(n/2^2)~~ + 2 * n/2
~~2^2*T(n/2^2)~~ = ~~2^3T(n/2^3)~~ + 2^2 * n/2^2
..
~~2^k*T(n/2^k)~~ = 2^(k+1)*T(n/2^(k+1))+2^k*n/2^k

T(n) = 2^(k+1) * T(n/2^(k+1)) + sum 2^i * n/2^i, i=0:k
n/2^(k+1) = 1 => T(n/2^(k+1)) = T(1) = Theta(1)
2^(k+1) = 2^log n = n
T(n) = n * Theta(1) + n * log n
T(n) = Theta(n + n* log n) =>
T(n) = Theta(n log n)

2. Metoda arborelui de recurenta



n/2^h = 1 => h = log n

=> T(n) = n * (h+1) = n * (log n +1) = n log n + n
=> T(n) = Theta(n log n)

3. Metoda substitutiei

-> se intuieste solutia/ se demonstreaza prin alta metoda
-> se demonstreaza prin inductie

alta metoda -> iterativa, arbore, Master

Master:
T(n) = a * T(n/b) + f(n)

n^ logb a ? f(n)

f(n) = O(n^(logb a – eps)) => T(n) = Theta (n^ logb a)
f(n) = Theta(n^ logb a) = T(n) = Theta (n^logb a * log n)
f(n) = W(n^(logb a + eps)) (si se verifica inca o conditie) => T(n) = Theta(f(n))

T(n) = 2 T(n/2) + k2 n, n>1
k1, n = 1

T(n) = Theta (n log n)

exista c1, c2, n0, a.i. oricare n>= n0

c1 * (n log n) <= T(n) <= c2 * (n log n)

caz de baza:
n=1: nu se poate

n=2:

c1 * 2 <= 2k1 + 2k2 <= c2 * 2 =>
c1 <= k1 + k2 <= c2 (1)

T(3) = 2 * T([3/2]) + k2 * n
daca lucram cu parte intreaga superioara -> T(3) = 2* T(2) + k2* n, e in regula, nu mai avem nevoie de alt caz de baza
daca lucram cu parte intreaga inferioara -> T(3) = 2* T(1) + k2*n => trebuie luat si 3 caz de baza

daca am fi avut T(n) = 3 T(n/3) + n => n=2 si n=3 cazuri de baza (chiar daca lucram cu parte intreaga superioara)

n=3:

c1 * 3 log 3 <= 2k1 + 3k2 <= c2* 3 log 3
c1 <= (2k1 +3k2)/ 3log3 <= c2 (2)

(cazul n=3 nu era necesar, deoarece in general se lucreaza cu parte intreaga superioara)

Ipoteza de inductie:
consideram adevarat pentru T(n/2) si demonstram pentru T(n)

c1* (n/2 log n/2) <= T(n/2) <= c2 * (n/2 log n/2) | * 2 + k2*n

c1* (n log n – n) + k2 n <= T(n) <= c2 * (n log n – n) + k2 n
c1 * n log n + n (k2 -c1) <= T(n) <= c2 n log n + n (k2 - c2)
vreau sa arat ca
c1 * n log n <= T(n) <= c2 * n log n

c1 * n log n <= c1 * n log n + n(k2 – c1) <= T(n) <= c2 n log n + n (k2 – c2) <= c2 * n log n

k2 – c1 >= 0
k2 – c2 <= 0
c1 <= k2 <= c2 (3)

c1 <= k1 + k2 <= c2 (1)
c1 <= (2k1 +3k2)/ 3log3 <= c2 (2)
c1 <= k2 <= c2 (3)

Din (1), (2) si (3) =>

c1 = min(k1+k2,(2k1+3k2)/3log3, k2)
c2 = max(k1+k2,(2k1+3k2)/3log3, k2)

Daca lucram cu parte intreaga superioara, cazul de baza n=3 nu mai este necesar =>
c1 = min (k1+k2, k2) = k2
c2 = max(k1+k2, k2) = k1 + k2

=> T(n) = Theta (n log n)

Consideram T(n) = Theta(n log n + n)
(observatie -> la master caz 2 e de preferat ca in loc de T(n) = Theta(n^klog n) sa alegem

T(n) = Theta(n^k log n + n^k)

caz de baza:
n=1:
c1 <= k1 <= c2 (1)

Ipoteza de inductie:

c1 *(n/2 log n/2 + n/2) <= T(n/2) <= c2 * (n/2 log n/2 + n/2) | *2 + k2n
c1 *(n log n – n + n) + k2 n <= T(n) <= c2 * (n log n – n + n) + k2 n
c1 * (n log n + n) + n (k2 – c1) <= T(n) <= c2 * (n log n + n) + n (k2 -c2)
c1 * (n log n +n) <= c1 * (n log n + n) + n (k2 – c1) <= T(n) <= c2 * (n log n + n) + n (k2 -c2) <= c2 * (n log n + n)

k2 – c1 >= 0
k2 – c2 <= 0
=>
c1 <= k2 <= c2 (2)

Din (1) si (2) =>
c1 = min(k1,k2)
c2 = max(k1,k2)

T(n) = T(n/2) + k2, n>1
k1, n=1

T(n) = Theta (log n + 1)
caz de baza:
n = 1:
c1 * 1 <= k1 <= c2 * 1
c1 <= k1 <= c2 (1)

Ip. de inductie:

c1* (log n/2 + 1) <= T(n/2) <= c2 * (log n/2+1) | + k2
c1 * (log n – 1 + 1) + k2 <= T(n) <= c2 * (log n -1 + 1) + k2
vreau sa ajung la
c1 * (log n +1) <=T(n) <= c2 * (log n +1)

c1 *(log n+1) <= c1 * (log n + 1) + (k2-c1) <= T(n) <= c2 * (log n +1) + (k2 – c2) <=c2 *(log n +1)

k2 -c1 >=0
k2 – c2 <= 0

c1 <= k2 <= c2 (2)

=> din (1) si (2)
c1 = min(k1, k2)
c2 = max(k1,k2)

Probleme la substitutie:
-> uneori nu se poate alege caz de baza n =1
-> uneori este de preferat ca in loc de complexitatea cea mai exacta (Theta (n^k), de exemplu) sa adaugam sau sa scadem ceva a.i. sa putem face calculele

T(n) = 5 T(n/3) + n

=> T(n) = Theta(n^log3 5)

c1*((n/3) ^log3 5) <= T(n/3) <= c2 * (n/3^ log3 5) | *5 + n

c1*(n^log3 5) <= c1* (n^log3 5) + n <= T(n) => nu se poate =>

T(n) = Theta(n^log3 5 - n)